

Name :- Mihir Mungara

Snowflake Assignment-10 : Improve the Performance of a Sample Dataset and Cost Management Strategy for Snowflake Usage

Snowflake Performance Optimization and Cost Management

Objective

The objective of this exercise is to analyze query performance using Snowflake Query Profile, optimize query execution using filters and clustering keys, and develop a cost management strategy for efficient Snowflake usage.

Part 1: Analyze and Improve Query Performance

Step 1: Create Sample Dataset

Create Customers Table

```
CREATE OR REPLACE TABLE CUSTOMERS (  
    CUSTOMER_ID INT,  
    CUSTOMER_NAME STRING,  
    CITY STRING  
);
```

Load Customers Data

```
INSERT INTO CUSTOMERS  
SELECT  
    SEQ4()+1,  
    'Customer_' || (SEQ4()+1),  
    CASE MOD(SEQ4(),5)  
        WHEN 0 THEN 'New York'  
        WHEN 1 THEN 'Chicago'  
        WHEN 2 THEN 'Dallas'  
        WHEN 3 THEN 'Boston'  
        ELSE 'Seattle'  
    END  
FROM TABLE(GENERATOR(ROWCOUNT => 100000));
```

Create Orders Table

```
CREATE OR REPLACE TABLE ORDERS (  
    ORDER_ID INT,  
    CUSTOMER_ID INT,  
    ORDER_AMOUNT NUMBER(10,2),  
    ORDER_DATE DATE  
);
```

Load Orders Data

```
INSERT INTO ORDERS  
SELECT  
    SEQ4()+1,  
    UNIFORM(1,100000,RANDOM()),  
    UNIFORM(100,5000,RANDOM()),  
    DATEADD(  
        DAY,  
        -UNIFORM(1,365,RANDOM()),  
        CURRENT_DATE()  
    )  
FROM TABLE(GENERATOR(ROWCOUNT => 1000000));
```

Step 2: Profile Initial Query

Disable cache:

```
ALTER SESSION SET USE_CACHED_RESULT = FALSE;
```

Execute query:

```
SELECT  
    c.CUSTOMER_NAME,  
    c.CITY,  
    SUM(o.ORDER_AMOUNT) AS TOTAL_SALES  
FROM CUSTOMERS c  
JOIN ORDERS o  
ON c.CUSTOMER_ID = o.CUSTOMER_ID  
GROUP BY  
    c.CUSTOMER_NAME,  
    c.CITY;
```

Query Profile Observations

- Full table scan on ORDERS table.
- Large join operation.
- Query Insight: Inefficient Aggregate.
- High processing overhead.

Metrics (Before Optimization)

Metric	Value
Rows Processed	1,000,000
Execution Time	1.7 sec
Query Insight	Inefficient Aggregate
Join Input	1,000,000 rows

Step 3: Apply Query Optimization

Optimization 1 – Add Filter

```
SELECT
    c.CUSTOMER_NAME,
    c.CITY,
    SUM(o.ORDER_AMOUNT) AS TOTAL_SALES
FROM CUSTOMERS c
JOIN ORDERS o
ON c.CUSTOMER_ID = o.CUSTOMER_ID
WHERE o.ORDER_DATE >= DATEADD(DAY, -30, CURRENT_DATE())
GROUP BY
    c.CUSTOMER_NAME,
    c.CITY;
```

Benefits

- Reduced rows scanned.
- Reduced join workload.
- Reduced aggregation cost.

Optimization 2 – Add Clustering Key

```
ALTER TABLE ORDERS
CLUSTER BY (ORDER_DATE);
```

Verify clustering:

```
SELECT SYSTEM$CLUSTERING_INFORMATION('ORDERS');
```

Benefits

- Improved micro-partition pruning.
- Faster filtering on ORDER_DATE.
- Reduced storage scanning.

Optimization 3 – Avoid Redundant Processing

Filter data before joins:

```
WITH FILTERED_ORDERS AS
(
    SELECT *
    FROM ORDERS
    WHERE ORDER_DATE >= DATEADD(DAY, -30, CURRENT_DATE())
)
SELECT
    c.CUSTOMER_NAME,
    c.CITY,
    SUM(f.ORDER_AMOUNT)
FROM CUSTOMERS c
JOIN FILTERED_ORDERS f
ON c.CUSTOMER_ID = f.CUSTOMER_ID
GROUP BY
    c.CUSTOMER_NAME,
    c.CITY;
```

Performance Comparison

Metric	Before	After
Rows Processed	1,000,000	~82,000
Execution Time	1.7 sec	1.4 sec
Join Cost	High	Reduced
Aggregation Cost	High	Reduced
Partition Pruning	None	Enabled

Conclusion

The optimized query reduced the number of rows participating in joins and aggregations. Applying a clustering key improved micro-partition pruning and reduced scan overhead.

Part 2: Snowflake Cost Management Strategy

Objective

Develop a strategy to monitor and optimize Snowflake compute costs.

1. Warehouse Management Best Practices

Use Auto Suspend

```
ALTER WAREHOUSE COMPUTE_WH
SET AUTO_SUSPEND = 60;
```

Warehouse automatically suspends after 60 seconds of inactivity.

Use Auto Resume

```
ALTER WAREHOUSE COMPUTE_WH
SET AUTO_RESUME = TRUE;
```

Warehouse resumes only when needed.

Use Appropriate Warehouse Size

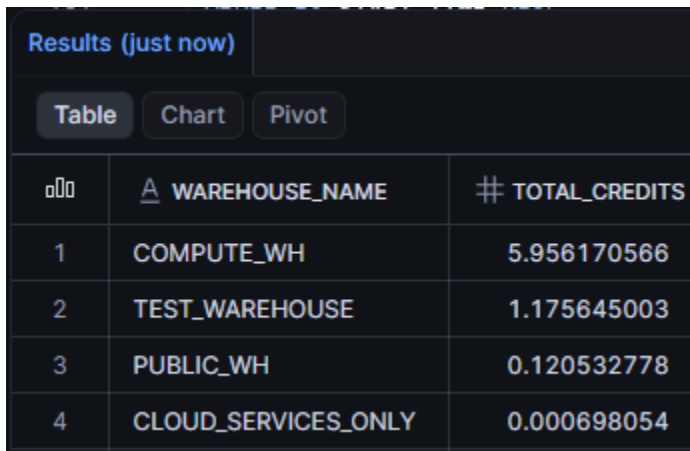
Workload	Warehouse Size
Development	XSMALL
Reporting	SMALL
Analytics	MEDIUM
Heavy ETL	LARGE

2. Monitor Warehouse Credit Usage

Warehouse Metering

```
SELECT
    WAREHOUSE_NAME,
    SUM(CREDITS_USED) AS TOTAL_CREDITS
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE START_TIME > CURRENT_TIMESTAMP() - INTERVAL '7 DAY'
GROUP BY WAREHOUSE_NAME
ORDER BY TOTAL_CREDITS DESC;
```

Output



The screenshot shows a Snowflake query results window. At the top, it says "Results (just now)". Below this are three tabs: "Table", "Chart", and "Pivot". The "Table" tab is selected. The table has three columns: a small icon column, a column labeled "WAREHOUSE_NAME", and a column labeled "TOTAL_CREDITS". There are four rows of data, numbered 1 to 4 in the first column.

	WAREHOUSE_NAME	TOTAL_CREDITS
1	COMPUTE_WH	5.956170566
2	TEST_WAREHOUSE	1.175645003
3	PUBLIC_WH	0.120532778
4	CLOUD_SERVICES_ONLY	0.000698054

3. Monitor Query Costs

Query History

```
SELECT
    QUERY_ID,
    USER_NAME,
    WAREHOUSE_NAME,
    EXECUTION_TIME
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
ORDER BY START_TIME DESC
LIMIT 20;
```

Identify

- Long-running queries
- Frequently executed queries
- Expensive warehouse usage

4. Reduce Query Costs

Add Filters

Instead of:

```
SELECT * FROM ORDERS;
```

Use:

```
SELECT *  
FROM ORDERS  
WHERE ORDER_DATE >= CURRENT_DATE()-30;
```

Select Required Columns Only

Instead of:

```
SELECT * FROM ORDERS;
```

Use:

```
SELECT ORDER_ID, ORDER_AMOUNT  
FROM ORDERS;
```

Use Clustering Keys

```
ALTER TABLE ORDERS  
CLUSTER BY (ORDER_DATE);
```

5. Scale Warehouses Based on Workload

Scale Up

For heavy ETL:

```
ALTER WAREHOUSE COMPUTE_WH  
SET WAREHOUSE_SIZE = 'LARGE';
```

Scale Down

After workload completion:

```
ALTER WAREHOUSE COMPUTE_WH  
SET WAREHOUSE_SIZE = 'XSMALL';
```

Example Cost Calculation

Assume:

Warehouse Size	Credits/Hour
XSMALL	1
SMALL	2
MEDIUM	4

Scenario

Warehouse runs for 2 hours:

MEDIUM Warehouse

Credits = 4×2

Credits Used = 8

If workload can run on SMALL:

SMALL Warehouse

Credits = 2×2

Credits Used = 4

Savings:

$8 - 4 = 4$ Credits

50% Cost Reduction

Cost Optimization Recommendations

1. Enable AUTO_SUSPEND and AUTO_RESUME.
2. Use the smallest warehouse that meets performance requirements.
3. Monitor WAREHOUSE_METERING_HISTORY regularly.
4. Optimize queries using filters and clustering keys.
5. Avoid unnecessary scans and joins.
6. Scale warehouses only when required.

Deliverables

SQL Scripts

Clustering Key

```
ALTER TABLE ORDERS  
CLUSTER BY (ORDER_DATE);
```

Warehouse Management

```
ALTER WAREHOUSE COMPUTE_WH  
SET AUTO_SUSPEND = 60;
```

```
ALTER WAREHOUSE COMPUTE_WH  
SET AUTO_RESUME = TRUE;
```

```
ALTER WAREHOUSE COMPUTE_WH SUSPEND;
```

Cost Monitoring

```
SELECT  
    WAREHOUSE_NAME,  
    SUM(CREDITS_USED)  
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY  
GROUP BY WAREHOUSE_NAME;
```

Outcome

Successfully analyzed query performance using Query Profile, optimized queries using filters and clustering keys, monitored warehouse credit consumption, and developed a cost management strategy to reduce Snowflake compute costs.